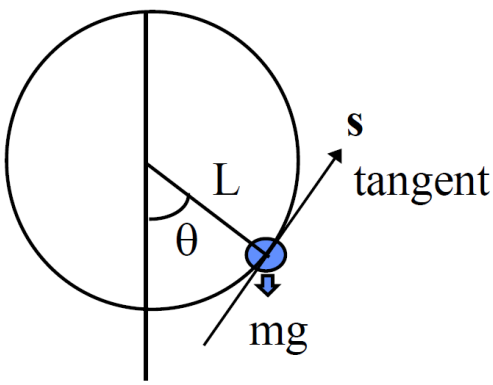
Lab 2: The Pendulum

Equation of motion for the nonlinear pendulum

The purpose of this laboratory is to teach you about dynamics of systems which have a nonlinear force law as well as how to use Python to solve coupled ordinary differential equations (ODEs).



The equation of motion for the pendulum is found from Newton's second law: we equating the mass times acceleration of the pendulum bob to the component of the force acting on the bob – its weight – along the direction of motion.

The bob moves on the arc of a circle of radius *L*, and the distance travelled along the tangent to the arc is denoted *s*. *L* is the length of the pendulum.

The distance travelled along the tangent to the arc traced out by the pendulum motion is

$$s = L\theta \qquad (1)$$

where θ is the angle subtended by the pendulum and a vertical line. θ is zero when the pendulum is at rest. The acceleration of the pendulum bob along the tangent is d^2s/dt^2. Since the length of the pendulum, *L*, is constant, the acceleration is

$$\frac{d^2 s}{d\,t^2} = L\,\frac{d^2\theta}{d\,t^2} \qquad (2)$$

The force which acts on the bob is its weight, *m g*. The component of the weight along the tangent is the force which accelerates the bob, - *m g* sin θ. The component of the weight along the pendulum string (or light rod) holding the bob, *m g* cos θ, is balanced by the tension in the string or rod and there is no acceleration in that direction.

The equation of motion for the pendulum, according to Newton's second law, is obtained by equating the mass times acceleration to the force acting along the tangent

$$m\,L\,\frac{d^2\theta}{d\,t^2} = -\,m\,g\,\sin\theta \qquad (3)$$

giving an equation of motion

$$\frac{d^2\theta}{d\,t^2} = -\frac{g}{L}\sin\theta \qquad (4)$$

The linear pendulum

For small angles, sin θ ~ θ , and equation (4) may be approximated by

$$\frac{d^2\theta}{d\,t^2} = -\frac{g}{L}\,\theta \qquad (5)$$

This is a *linearised* version of equation (4), since the right hand side is linear in θ whereas sin θ in equation (3) is nonlinear. When the pendulum equation of motion is approximated in this way it is referred to as the *linear* or *simple* pendulum.

Equation (5) can be solved analytically whereas equation (4) cannot. The solution to equation (5) is

$$\theta = B\,\sin(\omega_0\,t + \delta) \qquad (6)$$

where ω_0 is the natural frequency of oscillation and is equal to $\sqrt{g/L}$ and *B* and δ are the amplitude of motion and the initial phase, which both depend on the initial angle and angular velocity. Note that the frequency of oscillation does not depend on the amplitude of motion for the simple pendulum. This is not so, however, for the nonlinear pendulum in equation (4), where the sin θ term in the force is retained.

To solve a second order differential equation, such as equation (4) or (5) numerically, a new variable is introduced which transforms the second order problem into two first order problems. ω is the angular velocity of the bob, and allows us to rewrite the second order differential equation (5) as the following pair of first order equations:

$$\frac{d\,\theta}{d\,t} = \omega \qquad (7a)$$

$$\frac{d\,\omega}{d\,t} = -\frac{g}{L}\,\theta \qquad (7b)$$

Compare this with equation (5) and note that $\frac{d^2\theta}{d\,t^2} = \frac{d\,\omega}{d\,t}$. We make this transformation because it is much simpler to solve first order equations than second order equations numerically.

The damped, driven, nonlinear pendulum

So far we have only considered a pendulum bob moving under the force due to gravity, with or without a simplifying approximation sin θ ~ θ. The dynamics of the pendulum become more complex (and interesting) when we keep the nonlinear sin θ term in the weight of the pendulum bob and add terms which account for friction and an external driving force. If we add only a friction term then the pendulum quickly comes to rest. However, adding both friction and driving terms means that the pendulum does not come to rest and complex motion such as period doubling or chaos can be found.

The first order equations of motion when there are friction and driving forces are

$$\frac{d\,\theta}{d\,t} = \omega \qquad (8a)$$

$$\frac{d\,\omega}{d\,t} = f(\theta, \omega, t) \qquad (8b)$$

$$f(\theta, \omega, t) = -\frac{g}{L}\sin\theta - k\,\omega + A\,\cos\phi\,t \qquad (8c)$$

$f(\theta, \omega, t)$ has been introduced for convenience later. For convenience, we also choose the ratio $\frac{g}{L}$ in the inertial term to be unity. The damping term is $-\,k\,\omega$. The resistive force is proportional to the pendulum velocity and is in the opposite sense to the motion. The amplitude of the driving force is A. It is sinusoidal in time and has a frequency, ϕ. Initially, we chose *k* and *A* to be zero.

Solving ordinary differential equations by numerical methods

To solve these equations we need appropriate numerical methods. We choose initial values for θ and ω and step forward in time using a Taylor series expansion for both variables. The Taylor series for expansion of the function, *f*, about a point, *a*, is

$$f(a + h) = f(a) + h\,f'(a) + h^2/2!\,f''(a) + O(h^3) \qquad (9)$$

To solve our first order equations of motion for the linear pendulum we need to expand θ and ω about the current time:

$$\theta(t + \Delta t) = \theta(t) + \Delta t\,d\theta(t)/dt + O(\Delta t^2) \qquad (10a)$$

$$\omega(t + \Delta t) = \omega(t) + \Delta t\,d\omega(t)/dt + O(\Delta t^2) \qquad (10b)$$

Here Δt is the time step, and in this method we neglect terms of order Δt^2 or higher.

In order to get a sufficiently accurate solution to the equations of motion we must adopt a sufficiently small time step, Δt. A more convenient notation is now adopted for writing the equations of motion. Since time steps are discrete, they are labeled by an integer subscript, *n*. The equations of motion become:

$$\theta_{n+1} = \theta_n + \omega_n \Delta t \qquad (11a)$$
$$\omega_{n+1} = \omega_n + f(\theta_n, \omega_n, t)\, \Delta t \qquad (11b)$$

d θ(t)/dt has been replaced by $\omega = \omega_n$ and d ω(t)/dt by $f(\theta_n, \omega_n, t)$, which is simply equal to $-\theta$ in the case of the linear pendulum when g/L = 1 (see equation 8c, with sin θ replaced with θ, and *k* and *A* set to zero).

This method is called the *simple Euler method*. It is accurate to first order in time, and provides a simple introduction to numerical methods for solving ODEs. To solve the pendulum problem, we need a more accurate method which take higher order terms into account. The *trapezoid rule* is an improved method for solving ODEs, and replaces equations (11a) and (11b) with the following:

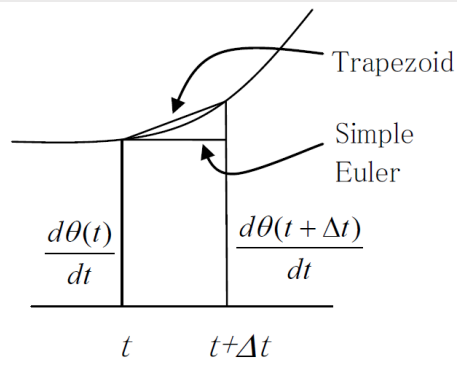
$$\theta_{n+1} = \theta_n + \omega_n \Delta t / 2 + (\omega_n + f(\theta_n, \omega_n, t)\, \Delta t)\, \Delta t / 2 \qquad (12a)$$
$$\omega_{n+1} = \omega_n + f(\theta_n, \omega_n, t)\, \Delta t / 2 + f(\theta_{n+1}, \omega_n + f(\theta_n, \omega_n, t)\, \Delta t), t + \Delta t)\, \Delta t / 2 \qquad (12b)$$

Details about how these expressions are derived are given on the following page.
To implement these equations, we can use a piece of code like the following:

```
k1a = dt * omega
k1b = dt * f(theta, omega, t)
k2a = dt * (omega + k1b)
k2b = dt * f(theta + k1a, omega + k1b, t + dt)

theta = theta + (k1a + k2a) / 2
omega = omega + (k1b + k2b ) / 2
t = t + dt
```

Derivation of trapezoid rule:



Trapezoid rule integration scheme

The area under the trapezoid in the figure is:

$$(t+\Delta t-t)\frac{\frac{d\theta(t)}{dt}+\frac{d\theta(t+\Delta t)}{dt}}{2} \cong \int_t^{t+\Delta t}\frac{d\theta}{dt}dt = \theta_{n+1}-\theta_n$$

According to the trapezoid rule, the change in θ over one time step is the time step, Δt, times the average value of the angular velocity at the beginning and the end of the time step. The angular velocity at the end of the time step is not known a priori and it must be estimated using a Taylor series

$$\theta_{n+1} \cong \theta_n + \frac{\Delta t}{2}\left(\frac{d\theta(t)}{dt}+\frac{d\theta(t+\Delta t)}{dt}\right)$$

$$\frac{d\theta(t+\Delta t)}{dt} = \frac{d\theta(t)}{dt}+\frac{d^2\theta(t)}{dt^2}\Delta t + O(\Delta t)^2$$

$$\frac{d\theta(t+\Delta t)}{dt} \cong \omega_n + f(\theta_n,\omega_n,t)\Delta t$$

$$\theta_{n+1} \cong \theta_n + \frac{\Delta t}{2}\left(\omega_n + \left(\omega_n + f(\theta_n,\omega_n,t)\right)\Delta t\right)$$

A similar calculation for the angular velocity gives

$$\omega_{n+1} \cong \omega_n + \frac{\Delta t}{2}\left(\frac{d\omega(t)}{dt}+\frac{d\omega(t+\Delta t)}{dt}\right)$$

$$\omega_{n+1} \cong \omega_n + \frac{\Delta t}{2}\left(f(\theta_n,\omega_n,t) + f(\theta_{n+1},\omega_n + f(\theta_n,\omega_n,t)\Delta t, t_{n+1})\right)$$

Exercise 1: Python script to solve the linear pendulum equation

1. Create a new python script / notebook / project in the editor of your choice.
   Import the numpy and matplotlib libraries.
   Create a function definition, f, corresponding to equation 8c, with three arguments: *theta*, *omega* and *t*.

2. Set the value of *k* to 0.0, ϕ *(phi)* to 0.66667 and *A* to 0.0 for the first exercise. Replace sin θ with θ. This will be used to solve the equation of motion for an undamped linear pendulum.

3. Initialise the values of *theta*, *omega*, *t*, *nsteps* and *dt* using the following code before your function definition for *f*:

```
theta = 0.2
omega = 0.0
t = 0.0
dt = 0.01
nsteps = 0
```

4. Check that your function works properly by getting the script to print values of *f* for several choices of values for *theta*, *omega* and *t*. Put a comment statement at the beginning of the script which tells the user what the script does.

5. Add the trapezoid rule equations (12a) and (12b) to your script after the function definition for *f*. You do not need to use separate variables for θ_n and θ_{n+1} and for ω_n and ω_{n+1}. Simply increment the current values of θ_n and ω_n using equations (12a) and (12b), as in the example code. Check that your script runs after implementing this step,

6. Enclose the trapezoid rule code in a *for* loop where the total number of steps taken is 1000. The time, *t*, should be incremented by a constant time step, *dt*, in the loop.
   The number of steps taken, *nsteps*, should be incremented by 1 in the loop.
   Check that your script runs after completing this step.

7. Add a plot command to plot θ and for ω at each time step. Remember that you need a `plt.show()` command *after* the *for* loop to see the final plot.
   Run the script and check that the plots of θ vs nsteps and ω vs nsteps are sinusoidal.

8. Place axis labels and a title on your graph. Add a `plt.axis()` command to your script so that the range on the x-axis is [0, 500] and the range on the y-axis is [-π, π]. The value of π is available in the *numpy* library as *np.pi*.

9. Use your script to make plots of *theta* versus *t* and *omega* versus *t* for the following

sets of initial conditions:

theta(0)	*omega*(0)
0.2	0.0
1.0	0.0
3.14	0.0
0.0	1.0

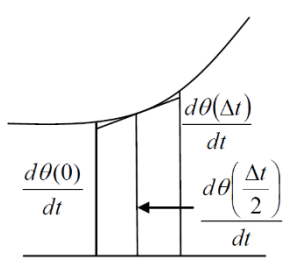
Exercise 2: Solving the nonlinear pendulum equation

1. Create a new copy of your simple pendulum code which you will modify to simulate the nonlinear pendulum. Name your file suitably, and add a comment statement which tells the user what the script does.

2. Changing the definition of *f* in its function definition will allow you to change the type of system simulated, e.g. from linear pendulum to nonlinear pendulum. Add a function definition for the f function in equation (8c) in which *theta* is replaced by *np.sin(theta)*.

3. Run your code for both linear and nonlinear pendulums using the initial conditions in the Table in Exercise 1.
   Plot graphs in which *theta* versus *time* and *omega* versus *time* are compared for the simple and nonlinear pendulums. Include these graphs in your report, and explain what they show.

Runge-Kutta integration schemes



Second order Runge-Kutta integration scheme

The *second order Runge-Kutta* algorithm for solving ODEs is similar to the trapezoid rule, except that the Taylor series expansion is carried out about the mid-point of the time step, rather than the beginning. This choice has the advantage that it is accurate to order Δt^2 rather than Δt.

The *fourth order Runge-Kutta* method is based on a similar principle, but with the time step divided into quarters. We will not derive this method here, but the equations to update *theta* and *omega* in the fourth order Runge-Kutta scheme are given below:

```
k1a = dt * omega
k1b = dt * f(theta, omega, t)

k2a = dt * (omega + k1b/2)
k2b = dt * f(theta + k1a/2, omega + k1b/2, t + dt/2)

k3a = dt * (omega + k2b/2)
k3b = dt * f(theta + k2a/2, omega + k2b/2, t + dt/2)

k4a = dt * (omega + k3b)
k4b = dt * f(theta + k3a, omega + k3b, t + dt)


theta = theta + (k1a + 2*k2a + 2*k3a + k4a) / 6
omega = omega + (k1b + 2*k2b + 2*k3b + k4b) / 6
t = t + dt
```

Exercise 3: Change the numerical integration algorithm

1. Create a new copy of your nonlinear pendulum code, which you will modify to simulate the nonlinear pendulum using the fourth order Runge-Kutta algorithm.
   Name and comment the file accordingly.

2. Add the equations above to your script as Python code within a for loop so that the nonlinear pendulum equations are solved by this method instead.

3. Test your new algorithm by running the calculation with both the trapezoid rule and Runge-Kutta algorithms, using a single script for initial values *theta* = 3.1415, *omega* = 0.0. The script should plot theta versus time for both methods on one graph, to be included in your report.

Exercise 4: The damped nonlinear pendulum

In this exercise you will simulate the behaviour of a damped pendulum, i.e. one which is subject to friction forces as well as the force of its own weight. The simplest assumption about forces such as air resistance is that the damping force depends linearly on velocity and acts to oppose the motion.

1. Create a copy of your code from Exercise 4, and change the value of *k* in the function where *f* is defined (equation 8c) from 0.0 to 0.5 so that damping (i.e. friction) is turned on.

2. The script should plot *theta* versus time and *omega* versus *time* on separate graphs.

3. Use initial values of *theta* = 3.0 and *omega* = 0.0 and run the script.
   Include the graphs that are generated in your write up, and explain the shapes of the curves for *theta* and *omega* versus time.

Exercise 5: The damped, driven nonlinear pendulum

In this exercise a periodic external force is added to the force acting on the pendulum. This is like the force pushing a swing, except that the force acts continuously and it takes positive and negative values. In this case the motion may be periodic or it may be aperiodic. The driving force has a frequency, ϕ, and a period $2\pi / \phi$. This sets a base period for the pendulum. If the pendulum motion remains periodic, the motion must have the period of the driving force or an integer multiple of the driving force. Otherwise the pendulum motion is aperiodic – in this case its motion is said to be chaotic.

When a parameter such as the amplitude of the driving force, A, is changed the motion may remain periodic but the actual period may change. In the pendulum, period doubling of the base period occurs many times before aperiodic, chaotic motion is observed.

Evidently the motion of the damped, driven nonlinear pendulum is much more complex than even the damped pendulum. A convenient way of representing this motion is to plot the angle *theta* versus the angular velocity, *omega*. This is called a phase space plot (*phase portrait*). In the units we have chosen ($g/L = 1$), the phase portrait for the linear pendulum is a circle. A pendulum swing with a larger amplitude has a phase portrait which is a larger circle.

The phase portrait for the damped, driven, nonlinear pendulum in a periodic motion is a closed loop. This loop is called a *limit cycle* and represents the behaviour of the pendulum after it has reached steady state, periodic motion. For nearly every set of initial conditions, the initial motion of the pendulum is not on the limit cycle. The pendulum motion before steady state, periodic motion is reached is called *transient* motion. In this exercise you will omit the transient motion from the phase space plot by plotting points on the phase portrait only after steady state motion has been reached.

The simplest way to determine when steady state motion has been reached is by plotting the phase portrait. For the first set of parameters you are given for the damped, driven nonlinear pendulum ($A = 0.9$, $k = 0.5$, $\phi = 0.66667$) the phase portrait is a single loop. Any ‘tail’ on the loop is the transient motion. If your phase portrait has a tail you need to begin plotting points at a higher iteration number.

1. Copy your code for the damped, nonlinear pendulum to a new file with appropriate name and comment.

2. Change the value of A in the function where f is defined (equation (8c)) from 0.0 to 0.9 so that a sinusoidal driving force is turned on.

3. Initialise a variable *iteration_number = 0* near the beginning of the script. *iteration_number* should be incremented by 1 each time the Runge-Kutta algorithm updates the values of *theta* and *omega*.

4. Initialise a variable *transient = 5000* in the script before the loop to solve for *theta* and *omega*. Your script should plot a graph of *theta* versus *omega* only if *iteration_number* is greater than *transient*. If *transient = 5000* does not eliminate the transient motion, increase its value.

5. Run the code for the following parameter values and generate a phase portrait for each case. Include the phase portraits in your write up. Explain what each phase portrait says about the pendulum motion for each set of parameters:

   A = 0.90, 1.07, 1.35, 1.47 and 1.5; *k = 0.5, phi = 0.66667* (both fixed)

6. For some of these parameter values the motion is circulation rather than swinging back and forth, so that theta increases indefinitely. However, a pendulum angle of *theta* cannot be distinguished from an angle of theta + 2 π. *theta* is therefore restricted to lie in the range [-π , π] in this case. Here is a code snippet to use:

```
if (np.abs(theta) > np.pi):
    theta -= 2 * np.pi * np.abs(theta) / theta
```

   In some cases you may need to adjust the range [-π , π] to [-π + *a* , π + *a*] to obtain a limit cycle as a single piece, i.e. not cut in two at +π or -π. *a* is a value to be determined. You must think of a way to change the given code snippet to change the range in this way.

Supplementary Exercise: Generate a fractal, chaotic attractor

The motions of nonlinear dynamical systems such as the damped, driven pendulum can be aperiodic for some parameter ranges. For example, for the case

$$A = 1.5,\ k = 0.5,\ \phi = 0.66667$$

the pendulum motion is aperiodic and chaotic. The motion of a chaotic system has some generic features. One of these is that its trajectory has the property of sensitivity to initial conditions, i.e. if the initial value of *theta* or *omega* is changed by a small amount then a short time later the difference in the trajectories beginning from the slightly different initial conditions will grow greater exponentially in time.

In a damped, driven system such as the pendulum, the phase space trajectory remains in a restricted volume of phase space. The phase portraits you have plotted in the previous exercise are two dimensional plots of *theta* versus *omega*. In fact, the true dimension of the pendulum phase space is three. The third dimension is the phase of the driving force, $\phi * t$.

The trajectory in phase space for a given set of initial conditions does not intersect the trajectory at an earlier time since to do so would mean that the particular point of intersection had two possible future trajectories from one set of dynamical variables. Trajectories in the chaotic phase portrait for $A = 1.5$ appear to cross in two dimensions but do not cross if they are plotted in the three dimensional phase space, *theta, omega,* $\varphi * t$.

By taking a slice through the phase space trajectory at a given phase angle of the driving force the fractal nature of the trajectory can be seen. In order to take a slice through the phase space trajectory the *theta* and *omega* coordinates must be recorded each time the phase angle $\phi * t$ *modulo* 2π takes the same value.

1. Copy your script for the damped, driven nonlinear pendulum to a new file with appropriate name and comment description.

2. Change the time step, *dt*, so that one period of the driving force is divided into 1000 time steps. The period of the driving force is $2\pi / \phi$ so your time step should be $2\pi / 1000\ \phi$.

3. Change the script so that each thousandth *(theta, omega)* point is plotted on a graph of *theta* versus *omega*. The total number of iterations needed to fill in the chaotic phase portrait is much larger than needed to generate a limit cycle for periodic motion. Increase the maximum number of iterations to 10000 and more in steps of 10000 until you get a reasonably dense plot.

4. One of the most important characteristics of a fractal is that it has the property of self-similarity on different length scales. Observe the shape of the whole phase portrait. Its shape could be generated by stretching and folding a small mass over and over. Select a small region of the phase portrait and enlarge it using the magnify button on the plot window. You will see a similar but less dense structure in the magnified region. This is the self-similarity property of the fractal phase portrait of a chaotic system.